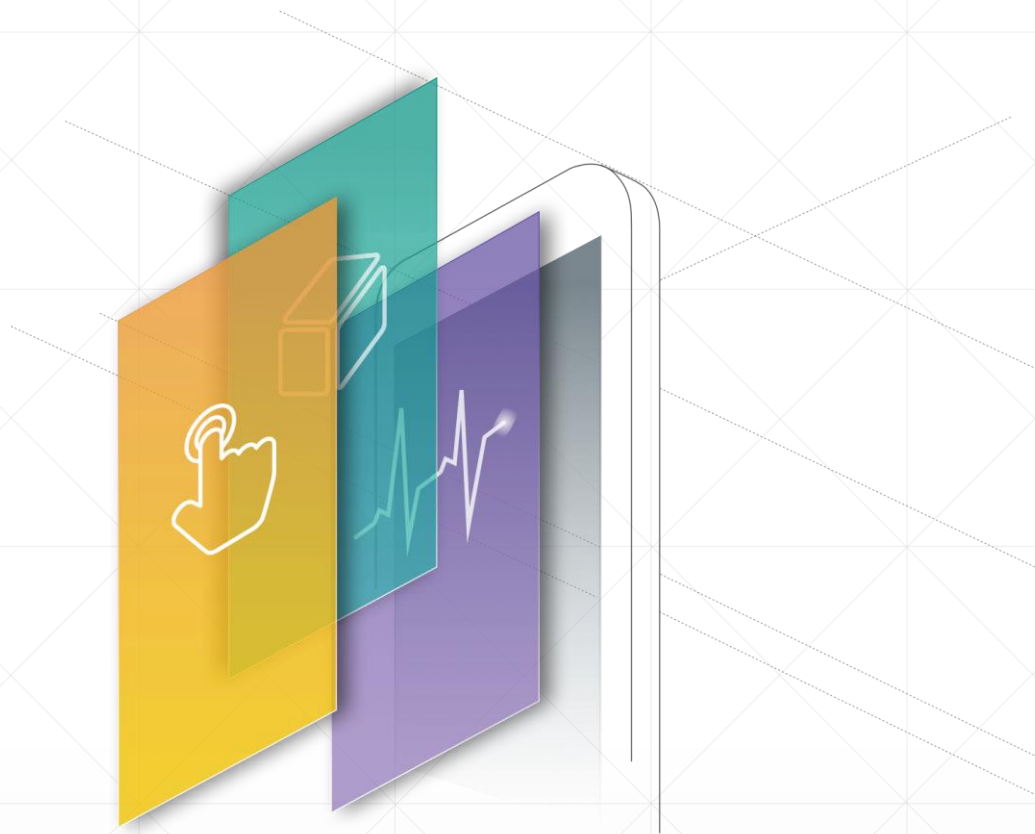


内部类

北京理工大学计算机学院
金旭亮



技术背景



- 与C++不一样，Java从一开始就设计为一个“纯”的面向对象语言，“一切皆为对象”。
- 基于这个理念，Java使用对象“封装一切”。
- 想象这样一个场景：一个类中包容许多方法和字段，有些字段和方法从逻辑上看具有比较紧密的联系，那么，把它们“放在一块”，“视为一个整体”，应该是一个很自然的想法。
- 针对以上场景，Java设计了“内部类”这个机制。

什么叫内部类 (inner class) ?



外部类

内部类

内部类的定义直接包容在另一个类内部。

在一个方法内部，也可以定义一个类，这也是“内部类”的一种类型。

关于内部类需要知道的.....



内部类可看成是外部类的成员，其地位等同于类中的其他成员，因此，它可以自由地访问外部的类成员（包括私有的成员）。



内部类编译以后，每个内部类都会产生一个.class文件，其文件名通常具有以下格式：

外部类名\$内部类名.class

内部类有几种表现形式，下面依次介绍……

静态内部类

一个声明为 static 的内部类示例：

```
class TopClass {  
    static class NestedClass{...}  
}
```

静态内部类的语法特性

```
class TopClass {  
    private static void func() {  
        System.out.println("TopClass.func");  
    }  
    private static int topClassValue = 100;  
    //静态内部类  
    static class NestedClass {  
        public void printInfo() {  
            System.out.println("静态内部类的实例方法。");  
            //存取外部类的静态字段  
            System.out.println("topClassValue = " + topClassValue);  
        }  
  
        public static void visit() {  
            //调用外部类的静态方法  
            func();  
            //存取外部类的静态字段  
            System.out.println(topClassValue);  
        }  
    }  
}
```

- 静态内部类不能访问其外部类的对象的实例数据，但可以访问其静态成员（包容字段与方法）。

```
public class StaticInnerClassTest {  
    public static void main(String[] args) {  
        //调用静态内部类的静态方法  
        TopClass.NestedClass.visit();  
        //实例化静态内部类对象，并调用它的实例方法  
        var obj = new TopClass.NestedClass();  
        obj.printInfo();  
    }  
}
```

参看：StaticInnerClassTest.java

静态内部类的使用场景



当需要编写许多静态方法及静态字段时，将逻辑上密切相关的代码组织成静态内部类，是一种合理的代码管理方式。

请看示例MinMax，其中的ArrayWithInnerClass类定义了一个静态minmax方法用于获取数组的最大值和最小值，为此，定义了一个静态内部类Pair，封装了这个信息，这么干了之后，外界使用起来就比较方便了，因它只需要处理一个Pair对象，而不需管理两个独立的变量。

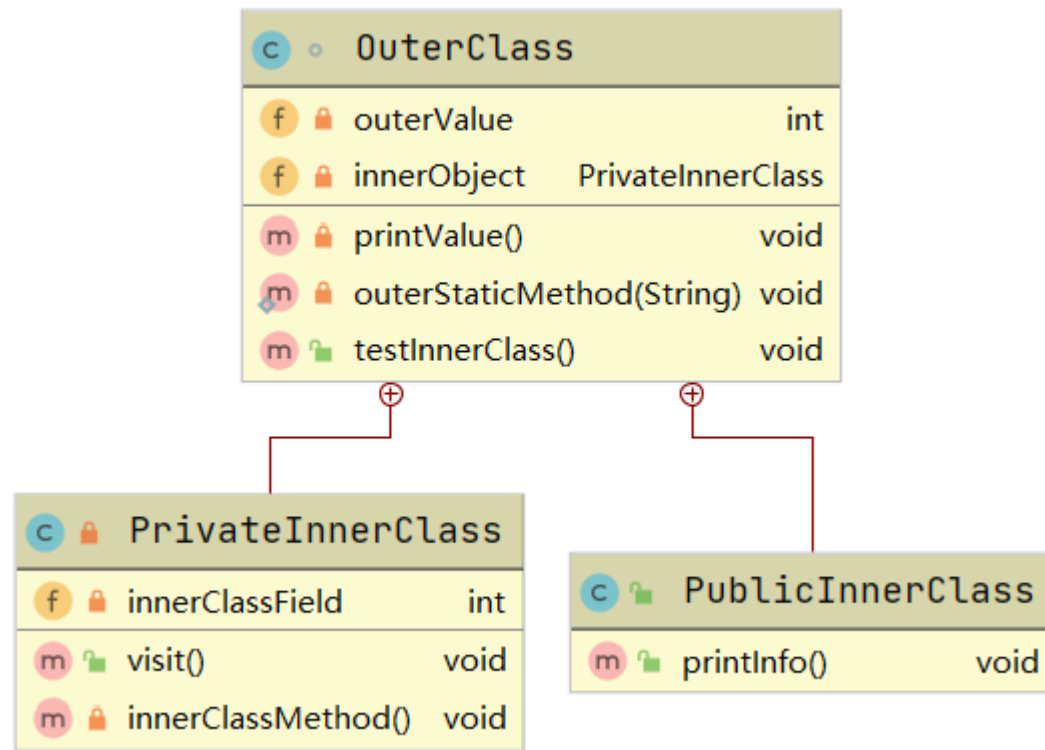


编写静态代码时，如果这些代码有可能在多线程环境下使用，则一定要注意线程安全问题，必要时需要加锁，但要注意这可能会带来性能损失。

成员内部类

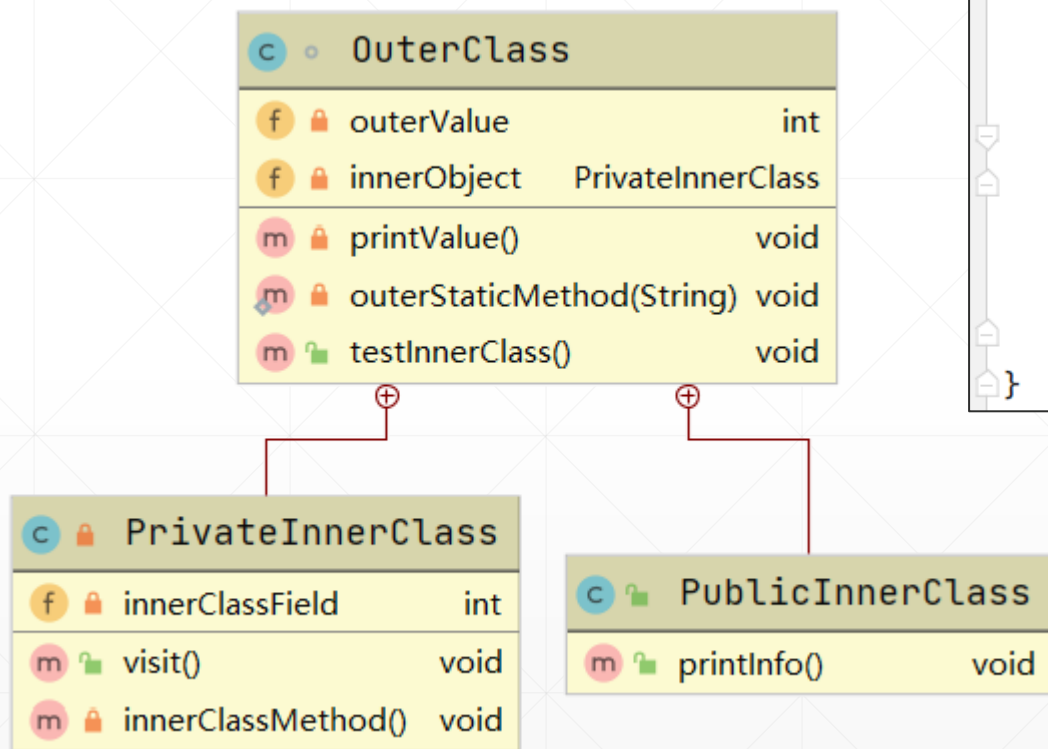
成员内部类，声明为类成员的非静态类。

```
class OuterClass {  
    class InnerClass {...}  
}
```



1. 内部类的声明不带有static关键字。它可以访问外部类中的任何成员。反过来，外部类也可以访问内部类的任何成员。
2. 在能够调用内部类的方法，以及为其数据字段赋值之前，外部类必须首先创建内部类的实例。

成员内部类的示例



```
public class MemberInnerClassTest {

    public static void main(String[] args) {
        //在实例化外部类时，私有的内部类对象也同时创建
        OuterClass outer = new OuterClass();
        //此方法在内部会调用私有内部类中的方法
        outer.testInnerClass();
        //公有的内部类，是可以直接在外部分实例化的
        //注意其独特的new方式
        OuterClass.PublicInnerClass obj = outer.new PublicInnerClass();
        obj.printInfo();
    }
}
```

示例：MemberInnerClassTest.java

注意一下公有/私有内部类不同的实例化方式。

成员内部类的使用场景

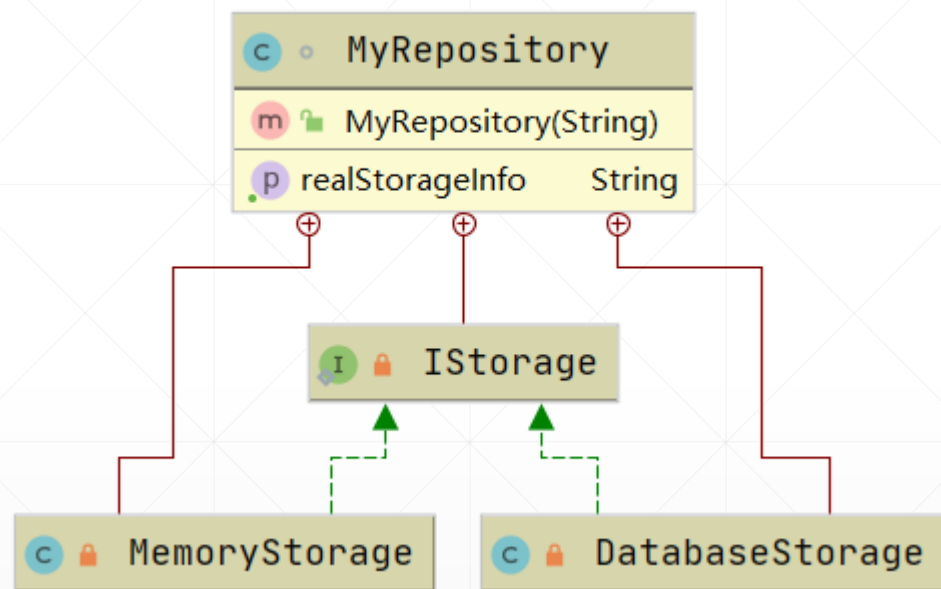


当一个类要完成的功能比较多，并且这些功能从逻辑上看很明显地可以划分为几大类，则将这些代码分别封装到几个成员内部类，是个不错的代码组织和管理方式。



通常情况下，成员内部类应该设置为 **private** 的，不允许外界直接访问它们，这是面向对象“封装”这一特性的具体应用实例。

成员内部类的应用实例



示例：UseMyRepository.java

某个可以用于保存数据的 `MyRepository` 类，其内部可以使用“内存集合”或“外部数据库”来保存数据。

具体使用哪种数据存储，在实例化 `MyRepository` 类时通过构造方法的参数确定，外界无需理会具体数据存储介质的技术细节，就能使用这个类。

MyRepository类的实现代码

```
class MyRepository {  
    private interface IStorage {  
        //一种存储介质需要实现的功能  
    }  
  
    //使用内存作为数据存储  
    private class MemoryStorage implements IStorage {  
    }  
  
    //使用数据库进行数据存储  
    private class DatabaseStorage implements IStorage {  
    }  
}
```

注意：IStorage和两个具体实现类都是私有的内部类型，外部无法直接访问

```
//真正使用的数据存储  
private IStorage realStorage;  
  
public MyRepository(String storageType) {  
    //依据构造方法传入的数值，实例化不同的数据存储介质  
    realStorage = switch (storageType.toUpperCase()) {  
        case "MEMORY" -> new MemoryStorage();  
        case "DATABASE" -> new DatabaseStorage();  
        default -> new MemoryStorage();  
    };  
}  
  
public String getRealStorageInfo() {  
    //供外界查询，到底使用的是哪种数据存储  
    return realStorage.getClass().getName();  
}  
}
```

MyRepository类的使用

```
public class UseMyRepository {  
    public static void main(String[] args) {  
  
        //我想使用内存作为数据存储  
        var memRepo = new MyRepository( storageType: "Memory");  
        //输出: MyRepository$MemoryStorage  
        System.out.println(memRepo.getRealStorageInfo());  
  
        //我想使用数据库作为数据存储  
        var dbRepo = new MyRepository( storageType: "Database");  
        //输出: MyRepository$DatabaseStorage  
        System.out.println(dbRepo.getRealStorageInfo());  
    }  
}
```

示例: UseMyRepository.java

静态内部类 vs. 成员内部类



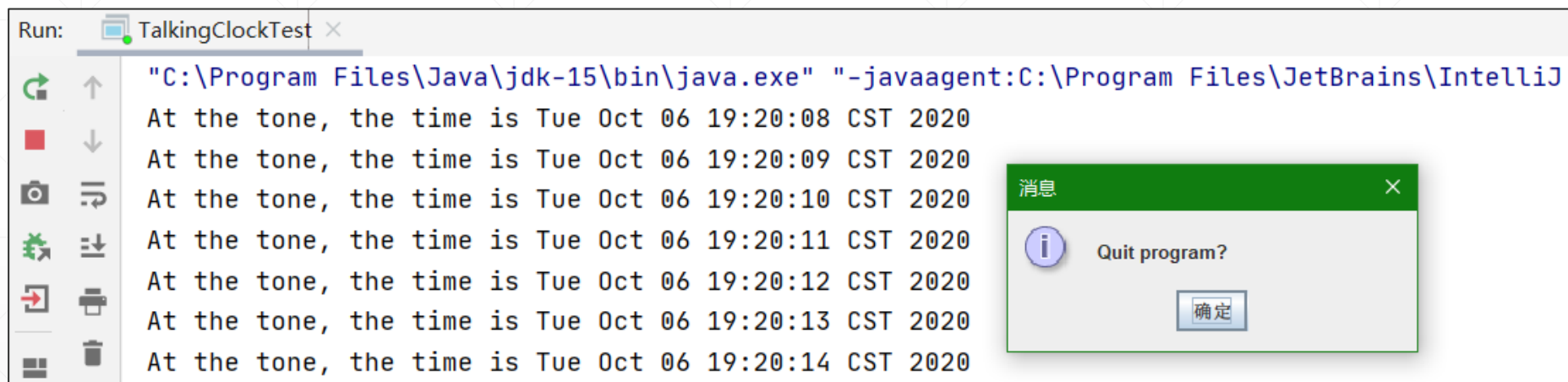
静态内部类，适合于封装那些与特定类型相关的公用代码，这些代码不依赖于外部的状态，也不依赖于具体的对象，最典型的例子，就是特定类型需要用到的特有“数学函数”。



成员内部类，主要用于分割代码，将相关的代码“打包”到一起，在类的内部实现代码重用。
成员内部类的代码，通常会直接访问其“外部”的实例数据字段或方法，是与特定的对象直接绑定的。

成员内部类的实例

一个能实现定时调用的自定义时钟类：



Run: TalkingClockTest x

```
"C:\Program Files\Java\jdk-15\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ"
At the tone, the time is Tue Oct 06 19:20:08 CST 2020
At the tone, the time is Tue Oct 06 19:20:09 CST 2020
At the tone, the time is Tue Oct 06 19:20:10 CST 2020
At the tone, the time is Tue Oct 06 19:20:11 CST 2020
At the tone, the time is Tue Oct 06 19:20:12 CST 2020
At the tone, the time is Tue Oct 06 19:20:13 CST 2020
At the tone, the time is Tue Oct 06 19:20:14 CST 2020
```

消息

Quit program?

确定

TalkingClockTest.java

定时闹钟示例分析

1. 示例在TalkingClock类内部定义了另一个TimePrinter类，然后将TimePrinter类的实例传给JDK中所提供的Timer对象，由它负责实现定时的调用——每隔一秒，输出当前时间。
2. 输出时间与消息框运行于两个独立的线程中，这是一个典型多线程应用程序。
3. 请仔细阅读示例源码，为以后的多线程内容学习打下基础。

TalkingClockTest.java

```
//一个可以定时输出当前时间的时钟类
class TalkingClock {

    public TalkingClock(int interval, boolean beep) {...}

    public void start() {
        //实例化一个TimerPrinter内部实例
        ActionListener listener = new TimePrinter();
        //以指定的时间间隔，调用listener对象事先约定好的方法
        Timer t = new Timer(interval, listener);
        t.start();
    }

    private final int interval; //信息打印间隔
    private final boolean beep; //是否鸣叫

    //内部类，完成时间打印任务
    private class TimePrinter implements ActionListener {...}
}
```

对象注入

成员内部类

本地内部类

在一个方法体中定义的类。

```
void foo() {  
    class Mylocal { ... };  
}
```



在方法体代码中，可以访问本地内部类的任何成员，本地内部类中的代码，也可以访问定义在方法中的局部变量。

本地内部类的示例

本地内部类在方法体内部定义一个类，然后实例化它，接着才能调用其方法，其好处主要是便于在一个方法内部重用代码。

```
public class LocalInnerClassTest {  
    public static void main(String[] args) {  
        TestLocalInnerClass();  
    }  
  
    //一个使用了本地内部类的静态方法  
    private static void TestLocalInnerClass() {  
        // 定义于方法体内部的内局部类  
        class MyLocalInnerClass {  
            private int field = 100;  
  
            private void printValue() {  
                System.out.println(field);  
            }  
        }  
        //本地内部类要实例化之后才能使用  
        var localObj = new MyLocalInnerClass();  
        localObj.printValue();  
    }  
}
```

LocalInnerClassTest.java

“匿名内部类”的引入

匿名内部类，是一种特殊的“本地内部类”，它的主要特点是——**没有名字!** 为了理解匿名内部类的语法，先来看一个引例。

以下代码定义了一个接口：

```
interface MyInterface {  
    void func();  
}
```

有一个方法，其参数为MyInterface类型，我们想调用它：

```
public void useMyInterfaceObject(MyInterface obj) {  
    obj.func();  
}
```

传统编程方式

通常情况下，我们必须先创建一个类实现MyInterface接口：

```
class MyInterfaceImple implements MyInterface{  
    @Override  
    public void func() {  
        System.out.println("实现了MyInterface接口的类运行");  
    }  
}
```

再new一个实现了接口的这个类的实例，之后才能调用前述需要使用接口对象作为参数的那个方法。

```
//经典的编程模式  
private static void useTraditionalClass() {  
    var obj = new AnonymousInnerClassTest();  
    var interfaceImplObj = new MyInterfaceImple();  
    obj.useMyInterfaceObject(interfaceImplObj);  
}
```

传统方式的问题.....



“先定义类，再new对象，接着调用方法”，这种方式虽然简单直接，但实在太啰嗦了，让人厌烦。



在比较大的项目中，有些代码仅仅只在很有限的范围内使用，而且通常只会使用一次，对于这些代码，使用传统编程方式，你也必须设计一个单独的类，这样一来，积累多了，就会导致类数量上的“爆增”，类太多了，就降低了代码的可维护性。



使用匿名内部类，可以省去为那些“仅用一次”的代码定义单独的类的麻烦，简化代码，避免出现“类”数目过多的弊端。

定义“匿名内部类”

修改传统方法调用方式代码，在方法调用语句中直接声明一个实现指定接口的“没有名字”的内部类，并立即使用new关键字创建一个它的实例：

1

```
obj.useMyInterfaceObject(  
    new MyInterface(){  
        public void func() { }  
    }  
);
```

上述代码省去了定义单独的实现MyInterface类的步骤，另外，匿名内部类对象可以访问外部类的所有成员。

2

```
private int outerField = 100;  
public void InnerVisitOuterField() {  
    var innnerObj = new MyInterface() {  
        @Override  
        public void func() {  
            //访问外部类的私有字段  
            AnonymousInnerClassTest.this.outerField++;  
            System.out.println(outerField);  
        }  
    };  
    innnerObj.func();  
}
```

示例：AnonymousInnerClassTest.java

拥有基类的匿名内部类

- 不使用接口，而是使用Java最顶层的基类Object，我们可以随意定义并获得一个匿名内部类的实例并立即调用它的方法。

```
new Object() {  
    public void printInfo(String message) {  
        System.out.println(message);  
    }  
}.printInfo("Hello");
```



上述代码实际上是创建了基类是Object的一个匿名内部类对象并立即调用它的方法。这种方法在实际开发中用得不多。这里列出来只是让大家知道可以这么用。

小结：匿名内部类的使用场景



- 匿名内部类多用于创建一个实现了某接口的对象或派生自某抽象基类的对象，其特点是：这个对象“仅在这个地方用”
- 在拥有可视化界面的GUI应用中，大量使用匿名内部类，比如为按钮的点击编写事件响应代码。
- 特别地，Java 8中引入了Lambda表达式特性，可看成是“针对接口实现的匿名内部类”的进一步简化。
- Java匿名内部类的语法很常见也很重要，是本讲需要重点把握的内容。

复盘与总结



为什么面向对象的编程语言要提供内部类这个特性，它有什么好处？

参考回答



我们希望以面向对象的方式封装某些信息为一个类，而这些信息仅在一个地方被使用，这时，创建一个“独立的”和“公开的”类是不必要的，而且它也不利于信息隐藏。因此，在这种场景下选择内部类是合适的。



对于匿名内部类，实际上它展示了“把代码本身当作数据”的思想（因为整个代码本身成为了另一个方法的参数），这一特点后面被进一步简化完善为Lambda表达式，从而让Java编程从纯的面向对象风格转换为“面向对象+函数式编程”的混合风格。在后面介绍Java 8新特性——Lambda时还会有更详细的介绍。

动手动脑



请使用匿名内部类重写本讲介绍的定时调用TalkingClockTest.java示例

参考答案: TalkingClockTest2.java